Лабораторная работа

Студент: Шкуратов Василий Сергеевич, М8О-101БВ-26 (101 группа)

Цель работы

Развернуть внутренний сервис из трёх контейнеров: frontend, api и logger. Настроить общение контейнеров по сети Docker, чтобы frontend обращался к api по имени сервиса, а не по IP. Сохранить строки logger в named volume, чтобы данные остались после перезапуска контейнеров и база данных не понадобилась. Собрать свой образ frontend и отправить его на Docker Hub.

Мой ник на GitHub: VasiliyMLCreator. Мой ник на Docker Hub: texnovasiliy.

Этап 1. Свой образ и Docker Hub

1.1. Сборка образа

Я собираю свой образ на базе *nginx:alpine*. В *Dockerfile* я добавляю *curl*, конфиг nginx и свою HTML-страницу. Флаг *--no-cache* заставляет Docker выполнить все шаги заново. Сборка заканчивается успешно, образ получает имя *texnovasiliy/docker-lab-frontend:1.0*.

Первый шаг FROM помечен как CACHED, хотя я указываю *--no-cache*. Я узнал, что этот флаг отключает кэш только для шагов *Dockerfile*. Базовый образ *nginx:alpine* уже скачан на мой компьютер, поэтому Docker берёт его оттуда.

Команда: *docker build --no-cache -t texnovasiliy/docker-lab-frontend:1.0 ./frontend*

```
[vasiliy@MacBook-Air-Vasiliy-4 docker-lab % docker build --no-cache -t texnovasiliy/doc]
ker-lab-frontend:1.0 ./frontend
[+] Building 0.6s (9/9) FINISHED                                docker:desktop-linux
 => [internal] load build definition from Dockerfile                              0.0s
 => => transferring dockerfile: 181B                                              0.0s
 => [internal] load metadata for docker.io/library/nginx:alpine                   0.1s
 => [internal] load .dockerignore                                                 0.0s
 => => transferring context: 2B                                                   0.0s
 => CACHED [1/4] FROM docker.io/library/nginx:alpine@sha256:a3554c0d0b48177b141   0.0s
 => => resolve docker.io/library/nginx:alpine@sha256:a3554c0d0b48177b1413c26e01   0.0s
 => [internal] load build context                                                 0.0s
 => => transferring context: 62B                                                  0.0s
 => [2/4] RUN apk add --no-cache curl                                             0.3s
 => [3/4] COPY nginx.conf /etc/nginx/conf.d/default.conf                          0.0s
 => [4/4] COPY index.html /usr/share/nginx/html/index.html                        0.0s
 => exporting to image                                                            0.1s
 => => exporting layers                                                           0.0s
 => => exporting manifest sha256:95f3e2ffce141cb11e9995f31d578bc0add3bd19879e39   0.0s
 => => exporting config sha256:84a11573cccc16eb96b380189d49b96b9f1838fdbfbfeabe   0.0s
 => => exporting attestation manifest sha256:34391d052fda22754e8868615a3b1104fb   0.0s
 => => exporting manifest list sha256:6bb164569b2a27106b957f5149365d599716cfc34   0.0s
 => => naming to docker.io/texnovasiliy/docker-lab-frontend:1.0                   0.0s
 => => unpacking to docker.io/texnovasiliy/docker-lab-frontend:1.0                0.0s
vasiliy@MacBook-Air-Vasiliy-4 docker-lab %
```

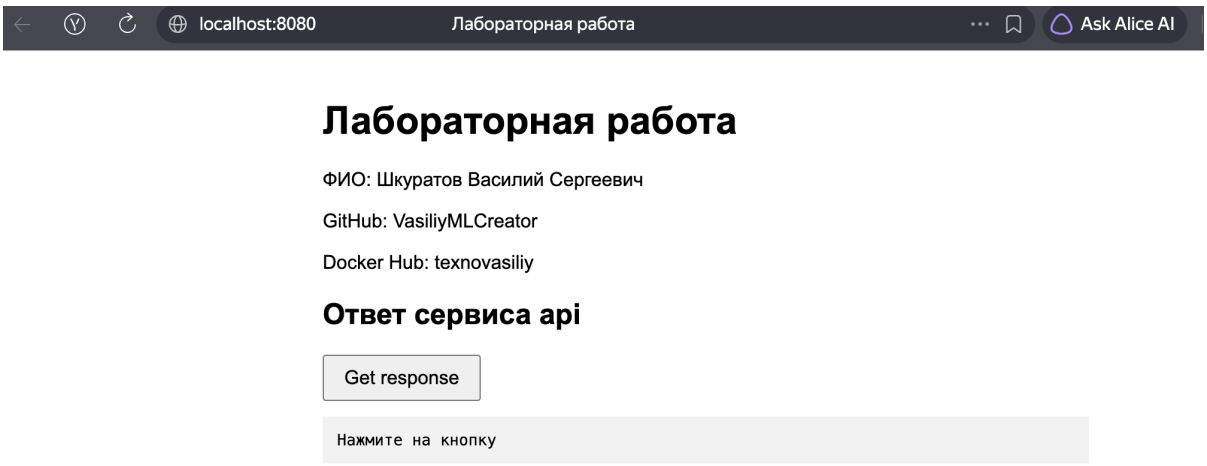
1.2. Запуск контейнера и страница в браузере

Я запускаю контейнер в фоновом режиме и пробрасываю порт 8080 моего компьютера на порт 80 контейнера. Docker печатает длинный идентификатор контейнера. Потом я открываю в браузере *http://localhost:8080*. На странице видны мои ФИО, ник на GitHub и ник на Docker Hub.

В параметре *-p 8080:80* левое число это порт хоста, а правое число это порт внутри контейнера. Nginx внутри слушает порт 80, а я открываю его снаружи через 8080.

Команда: *docker run -d --name frontend-test -p 8080:80 texnovasiliy/docker-lab-frontend:1.0*

```
[vasiliy@MacBook-Air-Vasiliy-4 docker-lab % docker run -d --name frontend-test -p 8080:]
80 texnovasiliy/docker-lab-frontend:1.0
b2635c5355d45249f4d454dd7508bfe09e051b18e48595b0408915cda044752e
vasiliy@MacBook-Air-Vasiliy-4 docker-lab %
```



1.3. Отправка образа на Docker Hub

Я вхожу в Docker Hub и отправляю образ. Первые попытки заканчиваются ошибкой EOF: соединение с Docker Hub обрывается. Потом Docker пишет insufficient scopes. Я узнал, что токен для входа должен иметь права Read & Write. С правами только на чтение *push* не проходит. Я создаю новый токен с нужными правами, и отправка проходит.

В выводе пять слоёв получают статус Pushed, а восемь слоёв получают статус Mounted from library/nginx. Docker не загружает заново слои базового образа nginx, если они уже есть на Docker Hub. Он загружает только мои слои. В конце Docker печатает digest sha256:6bb164569b2a...

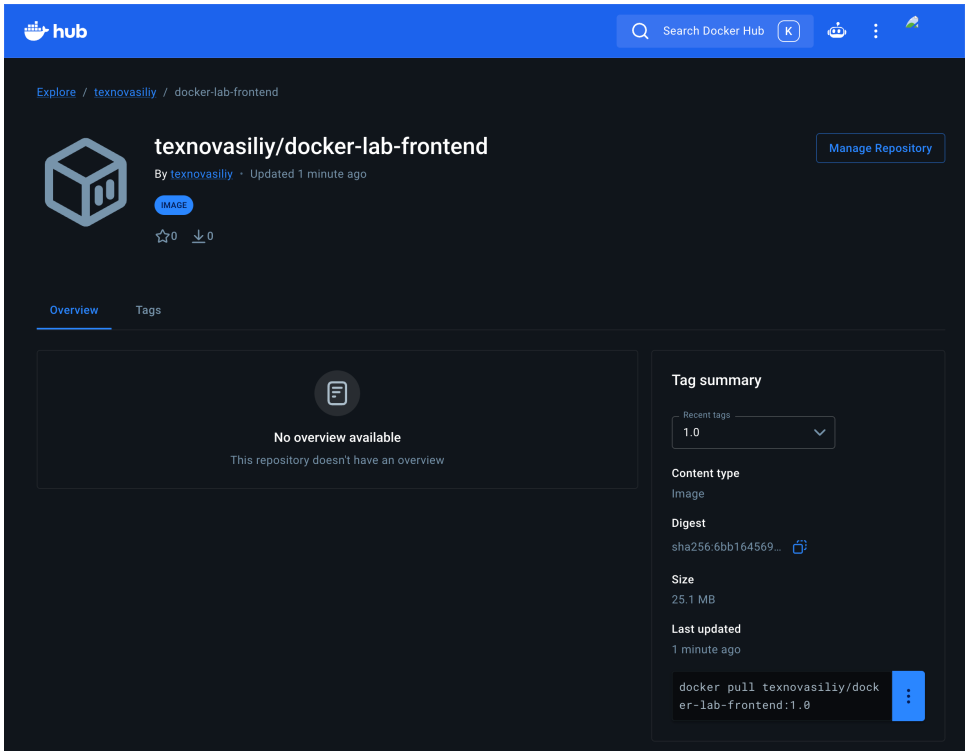
Команда: *docker push texnovasiliy/docker-lab-frontend:1.0*

```
[vasiliy@MacBook-Air-Vasiliy-4 docker-lab % docker push texnovasiliy/docker-lab-fronten]
d:1.0
The push refers to repository [docker.io/texnovasiliy/docker-lab-frontend]
1408044c7403: Mounted from library/nginx
d3531b7a4f21: Mounted from library/nginx
1d42ccbb2e1a: Pushed
44136fa355b3: Pushed
7b9d02243238: Mounted from library/nginx
f523b340a6c0: Mounted from library/nginx
dbb1ef19121a: Mounted from library/nginx
567a969e0221: Mounted from library/nginx
c2fefbef5314: Mounted from library/nginx
cbe54b031f3e: Mounted from library/nginx
8c586d118701: Pushed
a786b9c389ff: Pushed
3884c55459d3: Pushed
1.0: digest: sha256:6bb164569b2a27106b957f5149365d599716cfc34b43adaeeb3008f5292cf3f0 s
ize: 856
vasiliy@MacBook-Air-Vasiliy-4 docker-lab %
```

1.4. Образ на странице Docker Hub

Я открываю в браузере страницу репозитория texnovasiliy/docker-lab-frontend. На ней виден тег 1.0 и пометка Updated 1 minute ago. Digest на странице начинается с sha256:6bb164569, как в выводе *push*. Размер образа на Docker Hub равен 25.1 MB. Это размер сжатых слоёв, поэтому он меньше размера образа на диске.

Команда: открыть в браузере *https://hub.docker.com/r/texnovasiliy/docker-lab-frontend*



1.5. Удаление образа и повторное скачивание

Я удаляю локальный образ и скачиваю его с Docker Hub заново. На первом скриншоте команда *rmi* пишет Untagged и Deleted. Потом *pull* заканчивается ошибкой EOF на получении токена, потому что соединение с Docker Hub обрывается. Я повторяю только *pull* в цикле, и со следующей попытки он проходит. На втором скриншоте виден этот успешный *pull*, в конце Docker пишет Status: Downloaded newer image.

Digest после скачивания такой же, как до удаления: sha256:6bb164569b2a... Я узнал, что образ определяется своим содержимым. Если содержимое то же самое, то и digest тот же. Восемь слоёв получают статус Already exists, потому что они остались на компьютере от образа nginx. Пять слоёв я скачиваю заново: у трёх статус Pull complete, у двух Download complete.

Команда: *docker rmi texnovasiliy/docker-lab-frontend:1.0 && docker pull texnovasiliy/docker-lab-frontend:1.0*

Команда: *for i in $(seq 1 20); do docker pull texnovasiliy/docker-lab-frontend:1.0 && break; echo "попытка $i не удалась"; sleep 3; done*

```
vasiliy@MacBook-Air-Vasiliy-4 docker-lab % docker rmi texnovasiliy/docker-lab-frontend:1.0 && docker pull tex
novasiliy/docker-lab-frontend:1.0
Untagged: texnovasiliy/docker-lab-frontend:1.0
Deleted: sha256:6bb164569b2a27106b957f5149365d599716cfc34b43adaeeb3008f5292cf3f0
Error response from daemon: failed to resolve reference "docker.io/texnovasiliy/docker-lab-frontend:1.0": fai
led to authorize: failed to fetch oauth token: Post "https://auth.docker.io/token": EOF
```

```
vasiliy@MacBook-Air-Vasiliy-4 docker-lab % for i in $(seq 1 20); do docker pull texnovasiliy/docker-lab-front
end:1.0 && break; echo "попытка $i не удалась"; sleep 3; done
1.0: Pulling from texnovasiliy/docker-lab-frontend
3884c55459d3: Pull complete
d3531b7a4f21: Already exists
567a969e0221: Already exists
1408044c7403: Already exists
7b9d02243238: Already exists
f523b340a6c0: Already exists
dbb1ef19121a: Already exists
cbe54b031f3e: Already exists
c2fefbef5314: Already exists
8c586d118701: Pull complete
a786b9c389ff: Pull complete
44136fa355b3: Download complete
1d42ccbb2e1a: Download complete
Digest: sha256:6bb164569b2a27106b957f5149365d599716cfc34b43adaeeb3008f5292cf3f0
Status: Downloaded newer image for texnovasiliy/docker-lab-frontend:1.0
docker.io/texnovasiliy/docker-lab-frontend:1.0
```

Этап 2. Docker Compose с сетью и томом

2.1. Файл *docker-compose.yml*

Я открываю файл *docker-compose.yml*. В нём три сервиса: frontend, api и logger. Frontend собирается из папки ./frontend и открывает порт *8080:80*. Api это готовый образ *containous/whoami*. Logger это готовый образ *busybox*, который в бесконечном цикле пишет время в файл */data/log.txt*. Все три сервиса подключены к сети internal_network. Том *logger_data* подключён только к logger.

В команде logger стоит *$$* вместо $. Я понял, что двойной доллар нужен для compose. Он не подставляет переменную сам, а передаёт *$(date ...)* в оболочку контейнера.

Команда: *cat docker-compose.yml*

```
vasiliy@MacBook-Air-Vasiliy-4 docker-lab % cat docker-compose.yml
services:
  frontend:
    build: ./frontend
    image: texnovasiliy/docker-lab-frontend:1.0
    ports:
      - "8080:80"
    depends_on:
      - api
    networks:
      - internal_network

  api:
    image: containous/whoami
    networks:
      - internal_network

  logger:
    image: busybox
    command: sh -c 'while true; do echo "$$(date +%Y-%m-%dT%H:%M:%S) logger is alive"
>> /data/log.txt; sleep 2; done'
    volumes:
      - logger_data:/data
    networks:
      - internal_network

networks:
  internal_network:

volumes:
  logger_data:
vasiliy@MacBook-Air-Vasiliy-4 docker-lab %
```

2.2. Запуск всех сервисов

Я запускаю все три сервиса одной командой. Compose создаёт сеть, том и три контейнера. В таблице *docker compose ps* все три сервиса имеют статус Up.

Compose выводит жёлтое предупреждение про платформу. Образ whoami собран под linux/amd64, а мой Mac работает на arm64. Docker всё равно запускает такой образ через эмуляцию.

Команда: *docker compose up -d && docker compose ps*

```
vasiliy@MacBook-Air-Vasiliy-4 docker-lab % docker compose up -d && docker compose ps
[+] up 6/6
 ✔ Network docker-lab_internal_network Created                                0.0s
 ✔ Volume docker-lab_logger_data       Created                                0.0s
 ✔ Container docker-lab-logger-1       Started                                0.2s
 ✔ Container docker-lab-api-1          Started                                0.3s
 ! api                                 The requested image's platform (linux/a... 0.0s
 ✔ Container docker-lab-frontend-1     Started                                0.4s
NAME                    IMAGE                                  COMMAND
  SERVICE    CREATED         STATUS                  PORTS
docker-lab-api-1        containous/whoami                      "/whoami"
  api        1 second ago    Up Less than a second   80/tcp
docker-lab-frontend-1   texnovasiliy/docker-lab-frontend:1.0   "/docker-entrypoint.…"
  frontend   1 second ago    Up Less than a second   0.0.0.0:8080->80/tcp, [::]:8080->8
0/tcp
docker-lab-logger-1     busybox                                "sh -c 'while true; …"
  logger     1 second ago    Up Less than a second
vasiliy@MacBook-Air-Vasiliy-4 docker-lab %
```

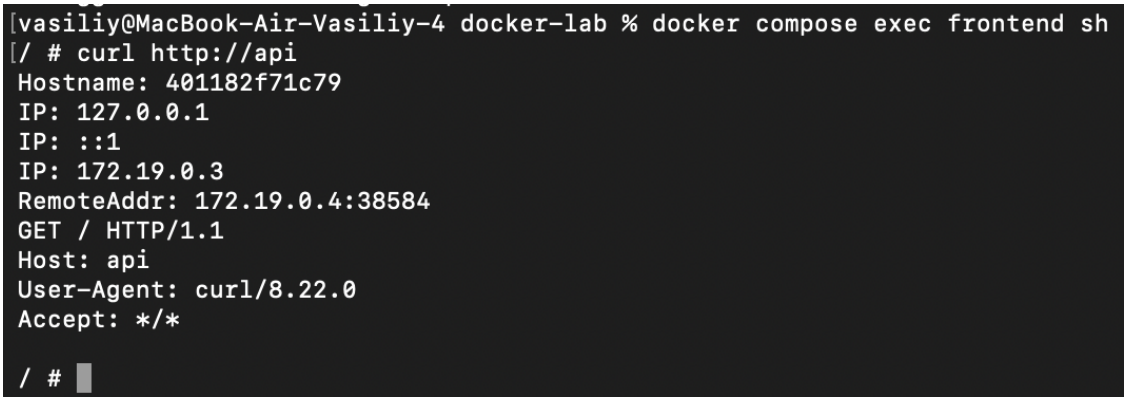
2.3. Обращение к api по имени сервиса

Я захожу в контейнер frontend и выполняю *curl http://api*. Приходит ответ от whoami. В нём указаны Hostname 401182f71c79, IP 172.19.0.3 и RemoteAddr 172.19.0.4. В заголовке Host стоит api. Значит, имя сервиса превращается в адрес контейнера.

Я понял, что за это отвечает встроенный DNS Docker. Адрес 172.19.0.3 принадлежит контейнеру api, а адрес 172.19.0.4 принадлежит контейнеру frontend, из которого идёт запрос. IP-адреса я нигде не пишу.

Команда: *docker compose exec frontend sh*

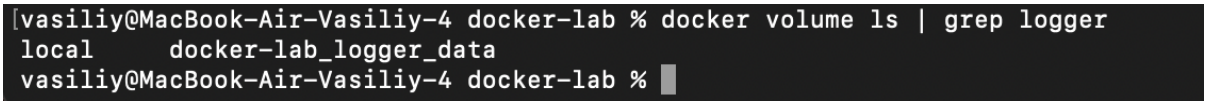
Команда: *curl http://api*



2.4. Том для logger

Я проверяю список томов. В нём есть том *docker-lab_logger_data*. Compose добавляет к имени тома префикс проекта. Префикс берётся из имени папки, поэтому *logger_data* превращается в *docker-lab_logger_data*.

Команда: *docker volume ls | grep logger*

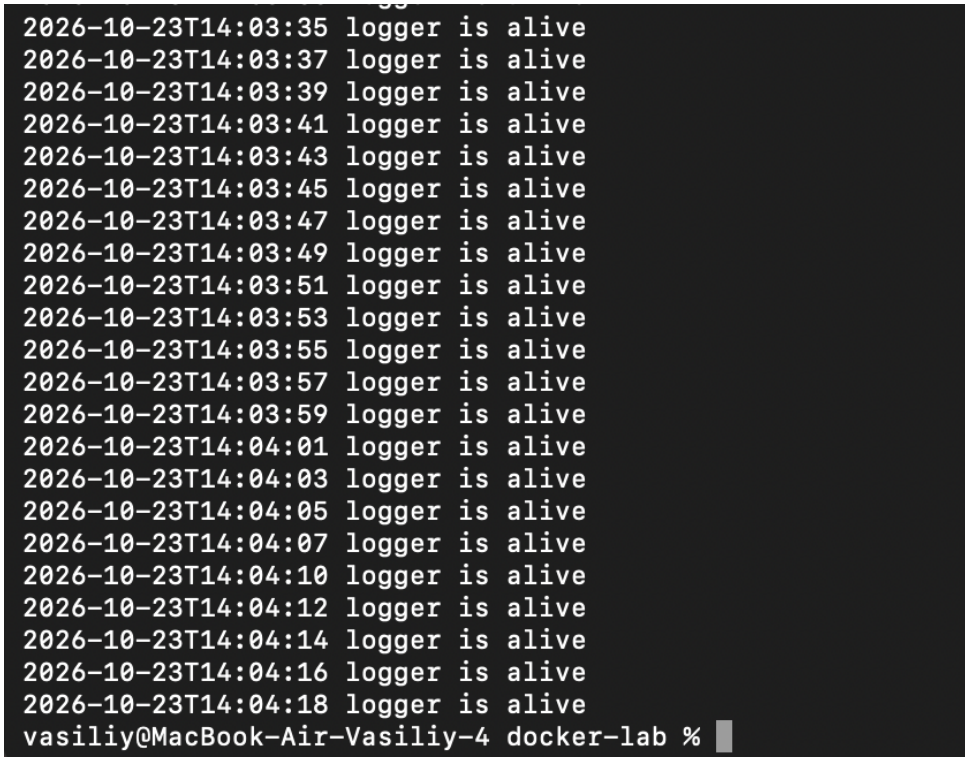


2.5. Строки, которые пишет logger

Я читаю файл */data/log.txt* внутри контейнера logger. В нём 98 строк, первая со временем 14:01:03, последняя со временем 14:04:18. На скриншоте виден конец вывода, потому что файл длинный.

Logger добавляет строку каждые две секунды. Один раз разница равна трём секундам: 14:04:07 и 14:04:10. К паузе *sleep 2* добавляется время работы самой команды, поэтому интервал иногда сдвигается.

Команда: *docker compose exec logger cat /data/log.txt*



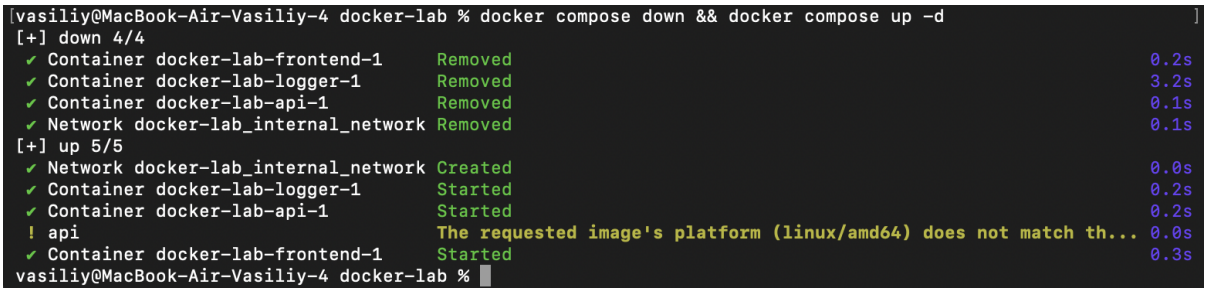
Этап 3. Том переживает контейнер

3.1. Пересоздание контейнеров без флага *-v*

Я останавливаю сервисы командой *down* и сразу запускаю их снова. *Down* удаляет три контейнера и сеть. В выводе четыре строки, и среди них нет тома. Потом *up* создаёт сеть и контейнеры заново, а том не создаёт, потому что он остался на месте.

Я узнал, что docker compose *down* без флага *-v* не трогает тома. Контейнеры одноразовые, а данные в томе живут отдельно.

Команда: *docker compose down && docker compose up -d*

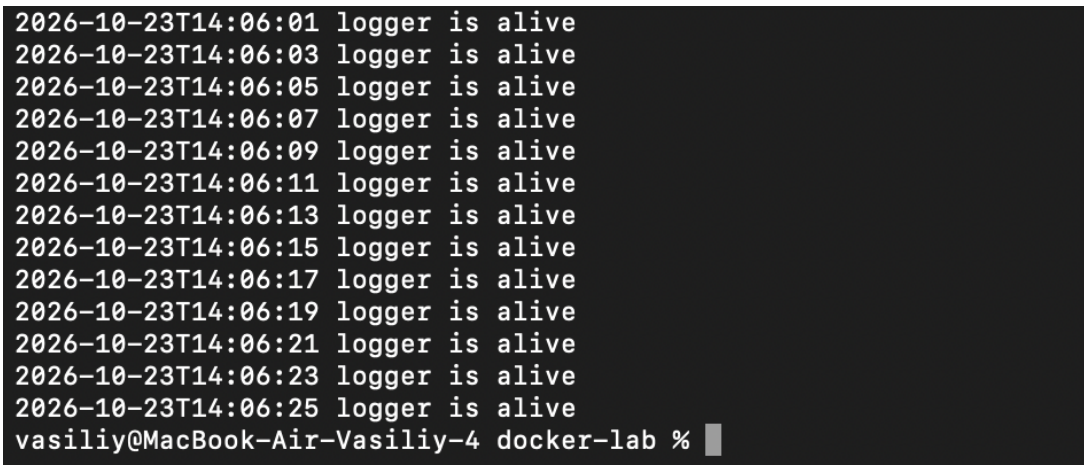


3.2. Файл логгера после пересоздания

Я снова читаю файл */data/log.txt*. Файл на месте, и в нём уже 162 строки вместо 98. Первая строка осталась прежней, 14:01:03, а последняя теперь 14:06:25. Значит, старые данные не пропали, а новые добавились в конец файла.

Контейнер logger создан заново, но он продолжает писать в тот же файл, потому что папка */data* смонтирована из тома.

Команда: *docker compose exec logger cat /data/log.txt*



3.3. Удаление тома флагом *-v*

Теперь я выполняю *down* с флагом *-v*. В выводе уже пять строк: среди них появилась строка Volume *docker-lab_logger_data* Removed. После этого *up* создаёт том заново, и *cat* показывает всего одну строку со временем 14:08:24. Второй скриншот я делаю чуть позже: в логе 15 строк, и все они новые, с 14:08:24 до 14:08:52.

Лог начинается с нуля. На шаге 2.5 первая строка имеет время 14:01:03. Том удаляется только вместе с флагом *-v*. Без этого флага данные переживают любое пересоздание контейнеров.

Команда: *docker compose down -v && docker compose up -d && docker compose exec logger sh -c "cat /data/log.txt"*

Команда: *docker compose exec logger sh -c "cat /data/log.txt"*

```
[vasiliy@MacBook-Air-Vasiliy-4 docker-lab % docker compose down -v && docker compose up -d && docker compose e]
xec logger sh -c "cat /data/log.txt"
[+] down 5/5
 ✔ Container docker-lab-logger-1       Removed                                                          3.1s
 ✔ Container docker-lab-frontend-1     Removed                                                          0.2s
 ✔ Container docker-lab-api-1          Removed                                                          0.1s
 ✔ Volume docker-lab_logger_data       Removed                                                          0.0s
 ✔ Network docker-lab_internal_network Removed                                                          0.1s
[+] up 6/6
 ✔ Volume docker-lab_logger_data       Created                                                          0.0s
 ✔ Network docker-lab_internal_network Created                                                          0.0s
 ✔ Container docker-lab-logger-1       Started                                                          0.2s
 ✔ Container docker-lab-api-1          Started                                                          0.2s
 ! api                                 The requested image's platform (linux/amd64) does not match th... 0.0s
 ✔ Container docker-lab-frontend-1     Started                                                          0.3s
2026-10-23T14:08:24 logger is alive
vasiliy@MacBook-Air-Vasiliy-4 docker-lab %
```

```
[vasiliy@MacBook-Air-Vasiliy-4 docker-lab % docker compose exec logger sh -c "cat /data/log.txt"
2026-10-23T14:08:24 logger is alive
2026-10-23T14:08:26 logger is alive
2026-10-23T14:08:28 logger is alive
2026-10-23T14:08:30 logger is alive
2026-10-23T14:08:32 logger is alive
2026-10-23T14:08:34 logger is alive
2026-10-23T14:08:36 logger is alive
2026-10-23T14:08:38 logger is alive
2026-10-23T14:08:40 logger is alive
2026-10-23T14:08:42 logger is alive
2026-10-23T14:08:44 logger is alive
2026-10-23T14:08:46 logger is alive
2026-10-23T14:08:48 logger is alive
2026-10-23T14:08:50 logger is alive
2026-10-23T14:08:52 logger is alive
vasiliy@MacBook-Air-Vasiliy-4 docker-lab %
```

Вывод

Я собрал свой образ на *nginx:alpine*, отправил его на Docker Hub и скачал обратно. Потом я поднял через compose три сервиса в одной сети. Я узнал, что внутри сети Docker сервисы находят друг друга по имени. Frontend обращался к api без IP-адреса.

Главное, что я понял: данные в named volume не зависели от контейнера. *Down* без флага *-v* удалял контейнеры и сеть, а том оставался вместе с логом. Только *down -v* удалял том, и лог начинался заново.